



Guía Azure DevOps

! INTRODUCCIÓN

La adopción de nuevas metodologías de trabajo requiere la definición de estándares y formas de aplicarla, pues de esta forma se garantiza que todos los equipos ejecuten las tareas de la misma forma. Este documento presenta las consideraciones técnicas, buenas prácticas y lineamientos definidos para la implementación de la metodología DevOps.



1. Lineamientos Generales



PRINCIPIOS FUNDAMENTALES

Los lineamientos generales para la implementación de DevOps consisten en el uso estandarizado de las herramientas de desarrollo para todos los proyectos nuevos. Para los proyectos antiguos se debe realizar un análisis y un plan de trabajo para establecer el estado objetivo y los pasos para lograrlo.

Las fases del ciclo de trabajo de DevOps están alineadas a la metodología de desarrollo de software y a las etapas del plan de acción, por lo tanto, no va en contravía de ningún otro lineamiento de desarrollo.



Figura 1. Ciclo de trabajo de DevOps.

Azure DevOps

La herramienta que soporta el ciclo de vida de DevOps es **Azure DevOps**, evolución de Team Foundation Services, utilizada como servicio en la nube. Sus funcionalidades incluyen:

-  Gestión de proyectos
-  Repositorio de código
-  Automatización de flujos de compilación
-  Despliegue (CI/CD)

ESTRUCTURA JERÁRQUICA

La herramienta tiene una estructura jerárquica de dos niveles principales:

1. **Organización:** Agrupa todos los proyectos de una entidad (GobernacionAntioquia)
2. **Proyectos:** Se crea uno por cada proyecto de desarrollo

 **Acceso:** <https://dev.azure.com/GobernacionAntioquia>

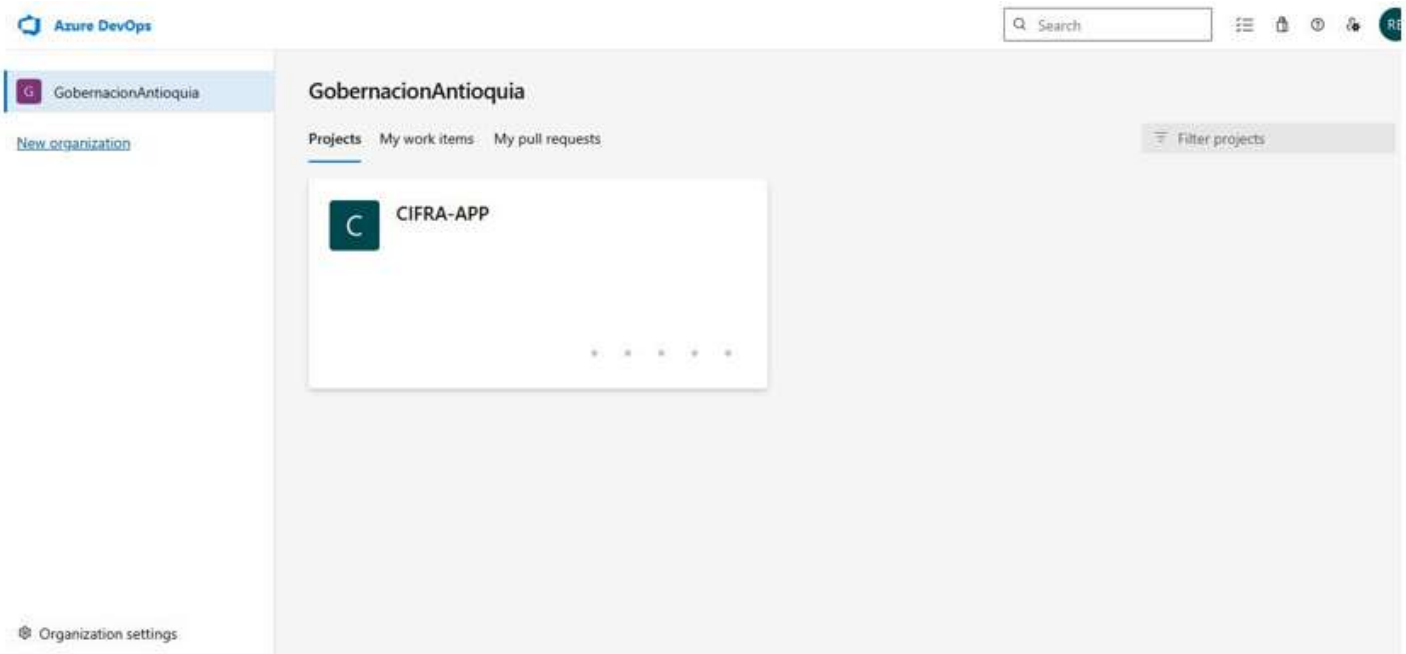


Figura 2. Vista de la organización.

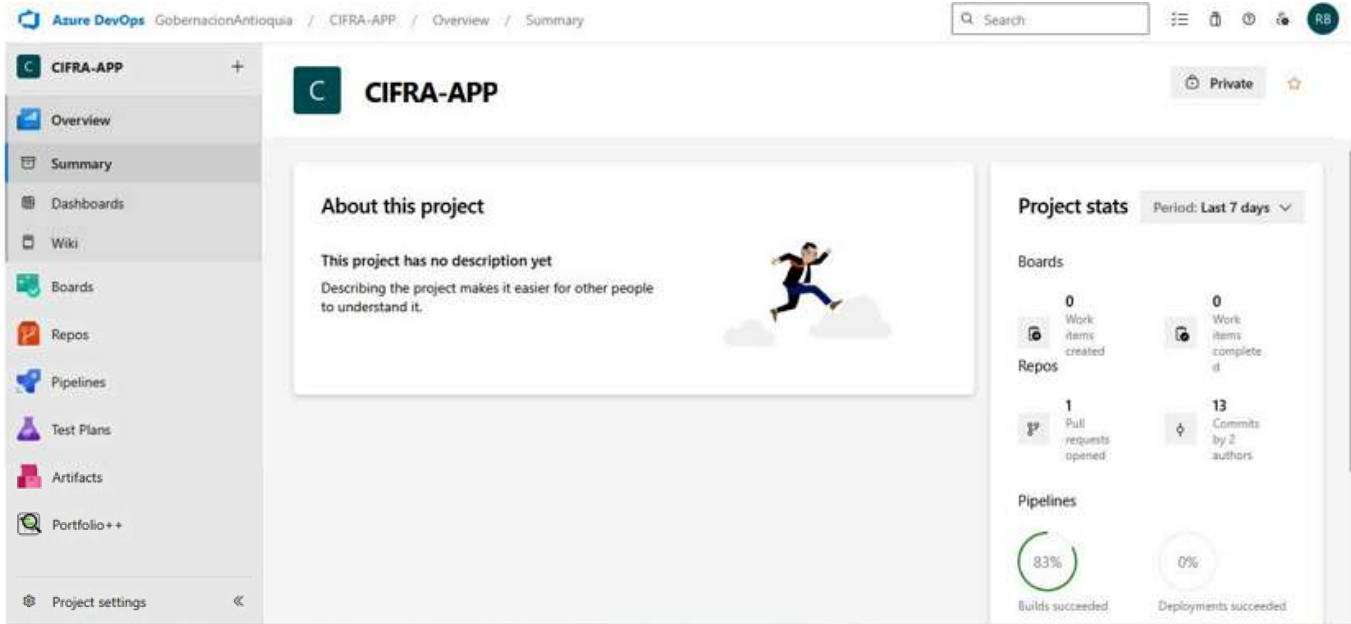


Figura 3. Vista de proyecto.

Quality Gates

IMPORTANCIA DE QUALITY GATES

Un Quality Gate es una medición de calidad que debe cumplir el software para su proceso de compilación y despliegue. Se establecen para garantizar la calidad de los procesos de desarrollo.

Métricas comunes incluyen:

-  Porcentaje de cubrimiento de pruebas automáticas
-  Deuda técnica por revisión estática de código
-  Vulnerabilidades de seguridad
-  Cumplimiento de plan de pruebas

IMPLEMENTACIÓN GRADUAL

Los Quality Gates se establecerán desde el comienzo, pero su nivel de exigencia se incrementará gradualmente para garantizar el cumplimiento.

Overview (Panel General del Proyecto)

El módulo Overview es la puerta de entrada al proyecto en Azure DevOps.

⚠️ BUENAS PRÁCTICAS

1. 📄 Project Description

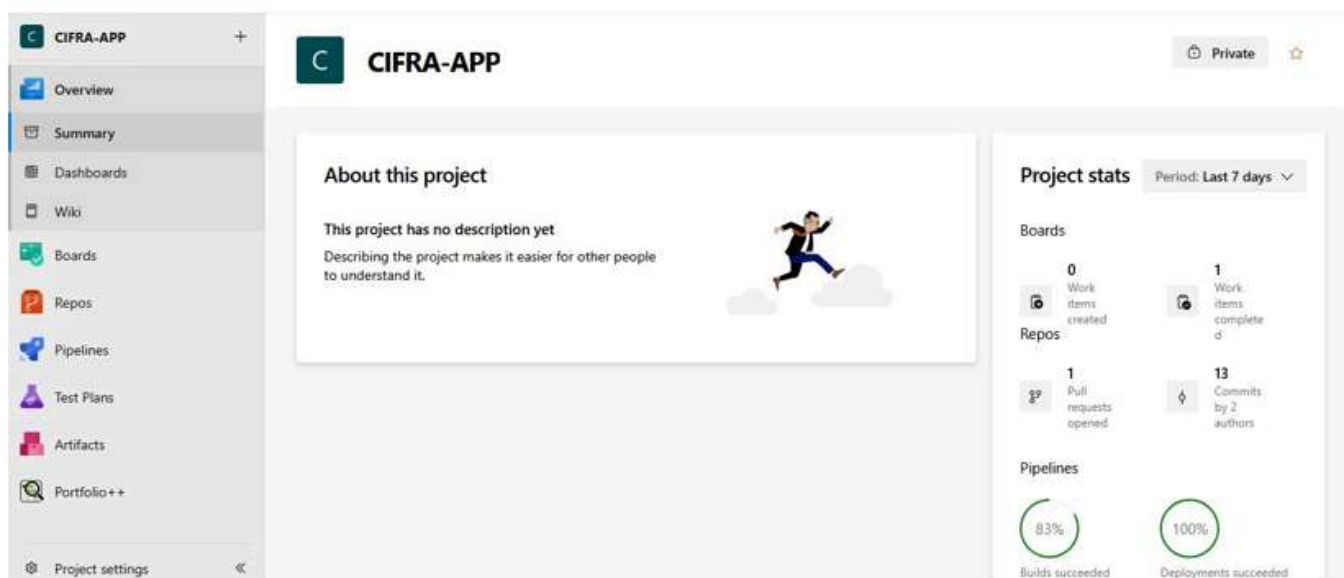
- Descripción clara y concisa del proyecto
- Incluir objetivos, alcance general y fechas clave
- Evitar tecnicismos para stakeholders no técnicos

2. 📊 Configuración del Dashboard Principal

- Incluir widgets de estado: Burn-down chart, Sprint Capacity, Work Item Status
- Personalizar para distintos roles: desarrollo, QA, gestión
- Actualizar periódicamente

3. 📌 Anclar Work Items o Queries relevantes

- Mostrar tareas críticas o recientes
- Mantener visible el progreso por iteración o epic

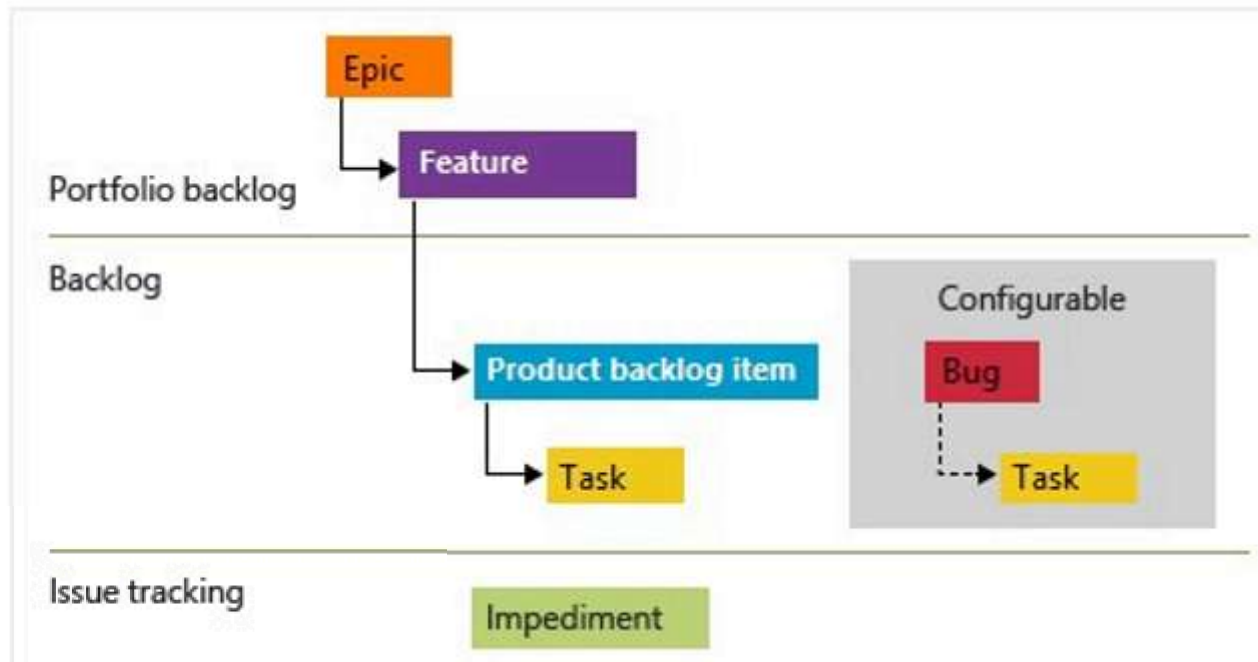


📋 Boards (Gestión Ágil del Trabajo)

ⓘ METODOLOGÍA SCRUM

El módulo Boards es esencial para la planificación, seguimiento y control del trabajo bajo metodologías ágiles, especialmente Scrum.

Jerarquía recomendada de Work Items:



1. Epic

- Iniciativas grandes desarrolladas en múltiples Sprints
- Representan funcionalidades o flujos completos de negocio

2. Feature

- Agrupaciones lógicas de historias de usuario
- Normalmente completadas en 1 a 3 sprints

3. User Story (PBI)

- Descripciones de funcionalidades desde la perspectiva del usuario
- Deberían completarse dentro de un solo sprint

4. Task

- Actividades técnicas para implementar una historia de usuario
- Asignadas directamente a miembros del equipo

5. Bug

- Pueden vincularse a User Stories o manejarse en paralelo
- Recomendable tratarlos como parte del backlog

BUENAS PRÁCTICAS EN BOARDS

1. Definición clara de la estructura del proyecto

- Utilizar Epics y Features para organizar el backlog
- No iniciar sprint sin User Stories definidos y priorizados

2. Uso adecuado de Tags y Areas

- Tags para categorizar por tecnología, módulo o prioridad
- Areas por equipo o componente funcional

3. Establecer Sprints y Iteraciones

- Definir iteraciones con fechas claras
- Asociar work items a cada sprint

4. Estimar usando Story Points

- Asignar puntos durante el backlog grooming
- Usar Fibonacci u otra escala consensuada

5. Revisar diariamente el Board

- Uso activo durante ceremonias Scrum
- Mantener estado actualizado: To Do, Doing, Done

Order	Work Item Type	Title	State
1	Proyecto	CIFRA Aplicación	New
	Objetivo	[CIFRA] R1.1 Mejorar Experiencia	New
	Épica	Diseño	New
	Historia de Usu...	[CIFRA] HU_119: Mejoras de experiencia de usuario...	Nuevo
	Task	Escenario 1	New
	Task	Escenario 2	New
	Task	Escenario 3	New
	Task	Escenario 4	New
	Task	Planteamiento nueva propuesta escenario 4	New
	Task	ESCENARIO 6	New



2. Codificación



ESTÁNDARES DE CODIFICACIÓN

El proceso de codificación debe alinearse a los estándares definidos para la construcción, garantizando consistencia y calidad en todos los proyectos.



2.1. Repositorio



OBLIGATORIO

Todos los proyectos de desarrollo de software (incluido mantenimiento) que soporten sistemas de información actuales o nuevos deben usar el repositorio GIT provisto en Azure DevOps.

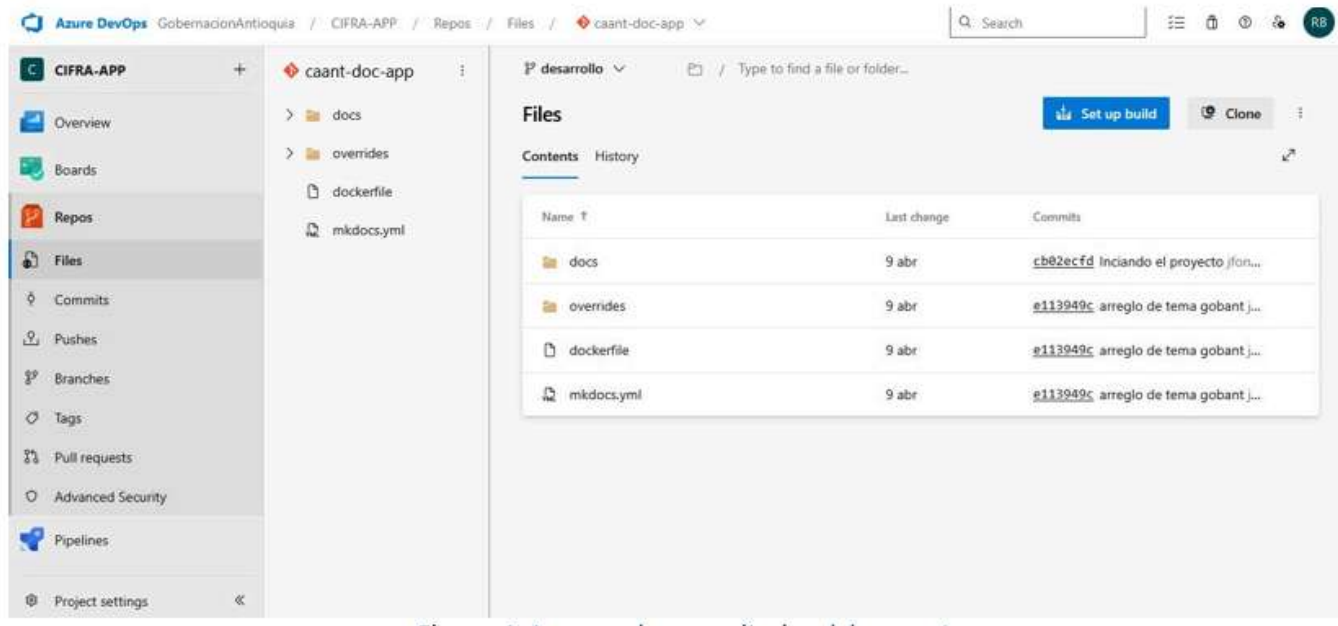


Figura 4. Acceso a los repositorios del proyecto.

NEW 2.1.1. Creación de repositorios

La creación se realiza desde la opción **Repos** usando la gestión de repositorios:

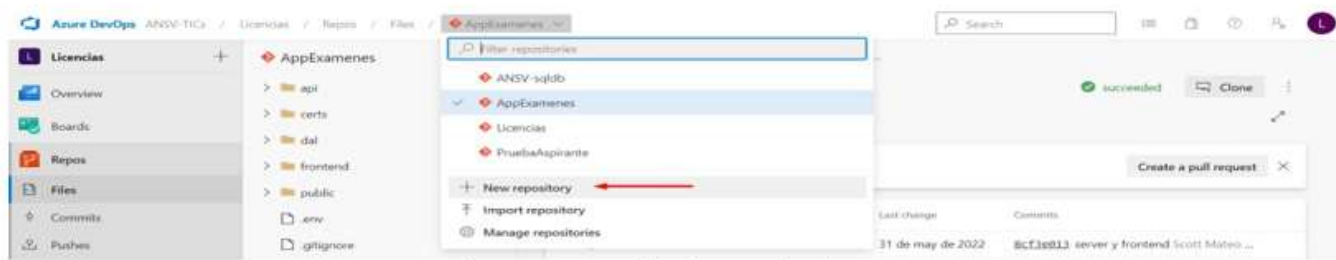
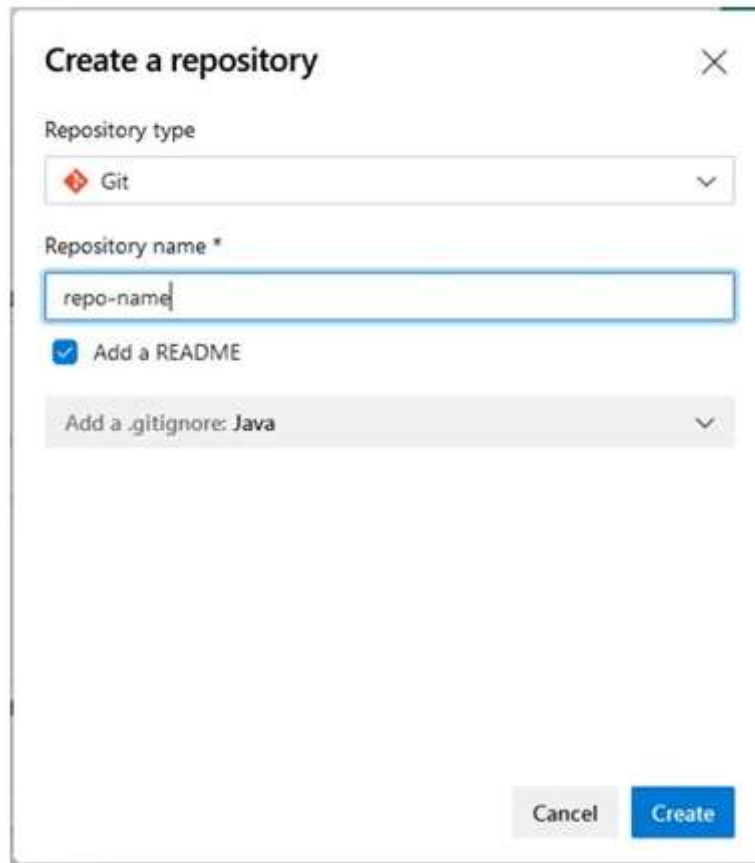


Figura 5. Creación de repositorio paso 1.



Create a repository

Repository type

Git

Repository name *

repo-name

☒ Add a README

Add a .gitignore: Java

Cancel Create

Figura 6. Creación de repositorio paso 2.

Campos requeridos:

- **Tipo:** GIT para todos los proyectos
- **Nombre:** Seguir lineamiento del punto 2.1.2
- **Add a README:** Seleccionar para documentación
- **Add a .gitignore:** Seleccionar lenguaje de desarrollo

¿CUÁNDO CREAR UN REPOSITORIO?

Cuando se necesite versionar código de cualquier componente que se deba desplegar independientemente: microservicio, desarrollo front, API backend, librería reutilizable, etc.

2.1.2. Nombramiento de repositorios

CRITERIOS DE NOMBRAMIENTO

Se debe crear el repositorio de acuerdo con sus características como alcance (si es específico del proyecto, del área o transversal a toda la entidad), tipo si es una librería, un proyecto

backend, o frontend, si es desarrollo en base de datos, microservicio, servicio de interoperabilidad entre otros (esta lista se refinará con la evolución de la metodología)

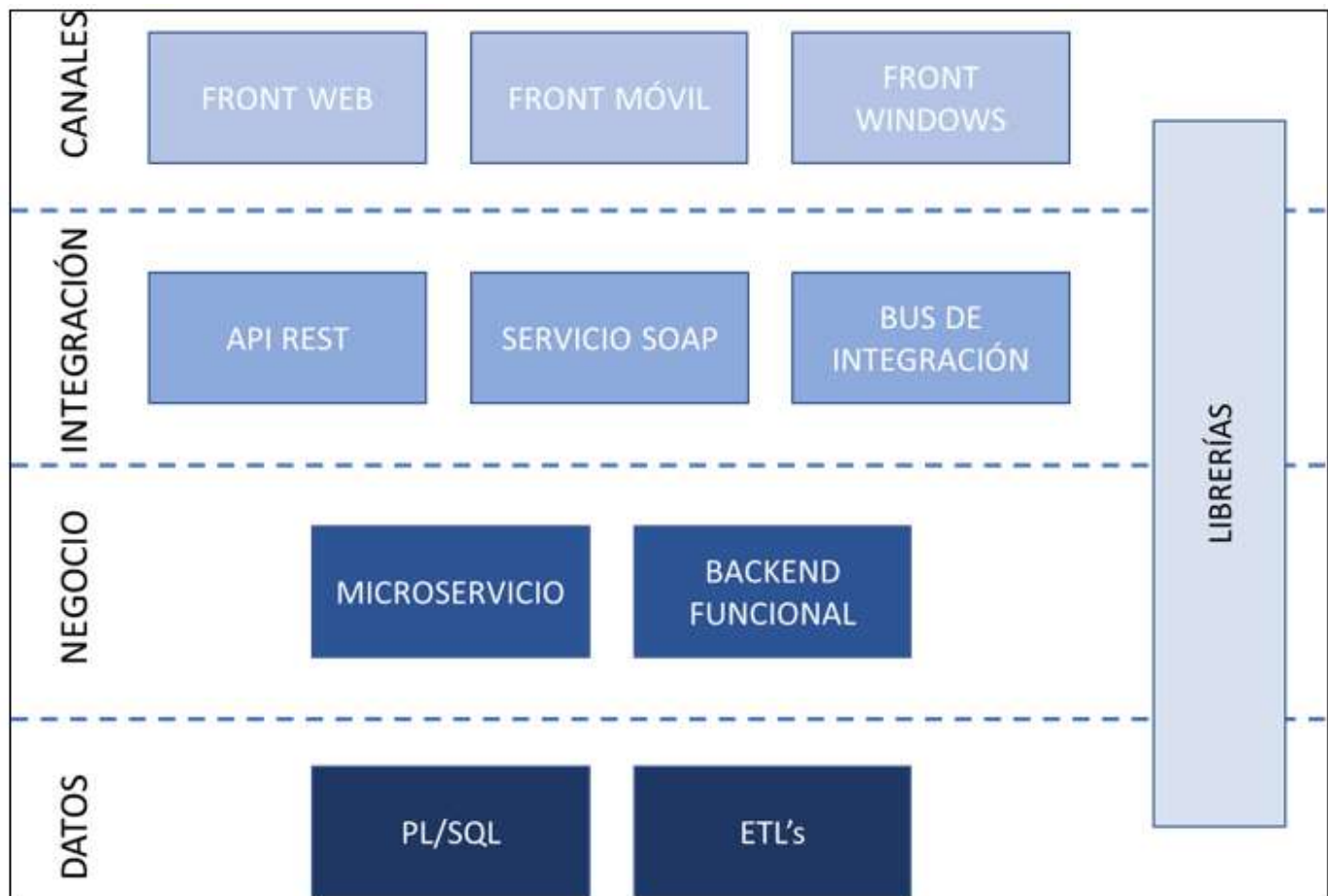
 Tabla de Buenas Prácticas:

Buena práctica	Descripción	Ejemplo correcto	Ejemplo incorrecto
 Nombre claro y descriptivo	Explica claramente el propósito	user-authentication-service	proyecto1
 Kebab-case o snake_case	Facilita lectura y evita errores	data_processor_api	DataProcessorAPI
 Evitar nombres genéricos	Nombres vagos dificultan mantenimiento	file-upload-service	test, new-app
 Incluir versión si aplica	Útil para múltiples versiones	mobile-app-v2	mobile-appnuevo
 Evitar información sensible	Protege seguridad y privacidad	internal-dashboard	cliente123passwords
 Incluir tecnología si es relevante	Identifica stack tecnológico	frontend-react, api-nodejs	frontend, backend
 Consistencia entre proyectos	Prefijos/sufijos comunes	platform-admin, platform-api	admin-panel, api-rest

 2.1.3. Alcance

 PRINCIPIO DE REUTILIZACIÓN

El alcance está dado por la capacidad de reutilización dentro de la entidad. El código no debe repetirse; si es reutilizable, crear repositorio según alcance.



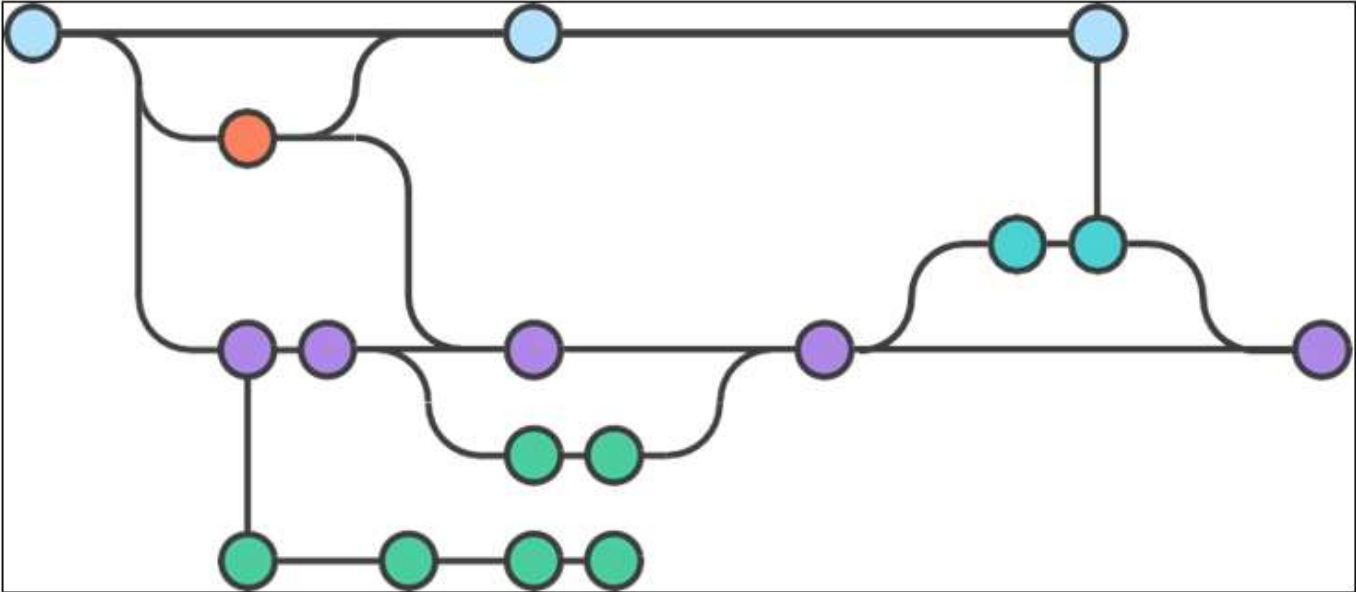
2.1.4. Descripción

- **Autónomo** por equipo de trabajo
- **Máximo 25 caracteres**
- Debe describir contenido/funcionalidad
- Separar palabras con guion "-"

2.1.5. GitFlow

💡 GITFLOW STANDARD

Para el manejo de ramas se usa el estándar **GIT Flow**, ampliamente usado en proyectos OpenSource y propietarios.



 Ramas principales (siempre existen):

- **Master:** Código estable desplegado en producción
- **Develop:** Integración de desarrollos, solo Pull Request

 Ramas auxiliares (necesidades específicas):

- **Feature:** Desarrollo individual de historias/correcciones
- **QAcalidad:** Versión previa a producción

 2.1.6. IDE

 HERRAMIENTAS RECOMENDADAS

Usar IDE moderno con integración GIT y soporte para SonarLint.

 Tabla IDE por Lenguaje:

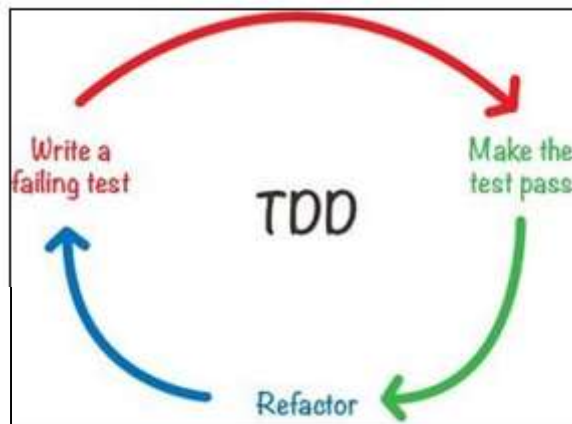
LENGUAJE	ECLIPSE	VS CODE	VISUAL STUDIO
 JAVA	 SI	 NO	 NO
 PHP	 NO	 SI	 NO

LENGUAJE	ECLIPSE	VS CODE	VISUAL STUDIO
 ANGULAR	✗ NO	✓ SI	✗ NO
 .NET (C#, VB)	✗ NO	✗ NO	✓ SI

2.1.7. TDD (Test Driven Development)

DESARROLLO GUIADO POR PRUEBAS

Estilo de desarrollo donde primero se diseña la prueba unitaria según requerimiento funcional, luego se desarrolla código hasta éxito, finalmente se refactoriza.



Aplicar en:

- Lógica backend
- Servicios web
- Utilidades y librerías
- Controladores MVC

Este estilo de desarrollo se debe usar para todo el código que sea susceptible de ser probado usando pruebas unitarias, como lógica en backend, servicios web, utilidades, librerías, controladores en MVC, etc; Su aplicación permitirá agilizar, simplificar y garantizar el uso de pruebas unitarias dentro de los procesos de construcción de software en la Gobernación de Antioquia.



3. Construcción

! INFO

La construcción de los artefactos de código es un mecanismo que permite habilitar la aplicación de la metodología DevOps, es por esto por lo que se estandariza el uso de una herramienta para versionar el código y construirlo, de tal forma que los flujos de construcción automatizados puedan descargar el código y correr la tarea de construcción sin intervención humana.



3.1. Clone

Evento para copiar repositorio remoto localmente. URL desde Azure DevOps → Repos → botón **Clone**.

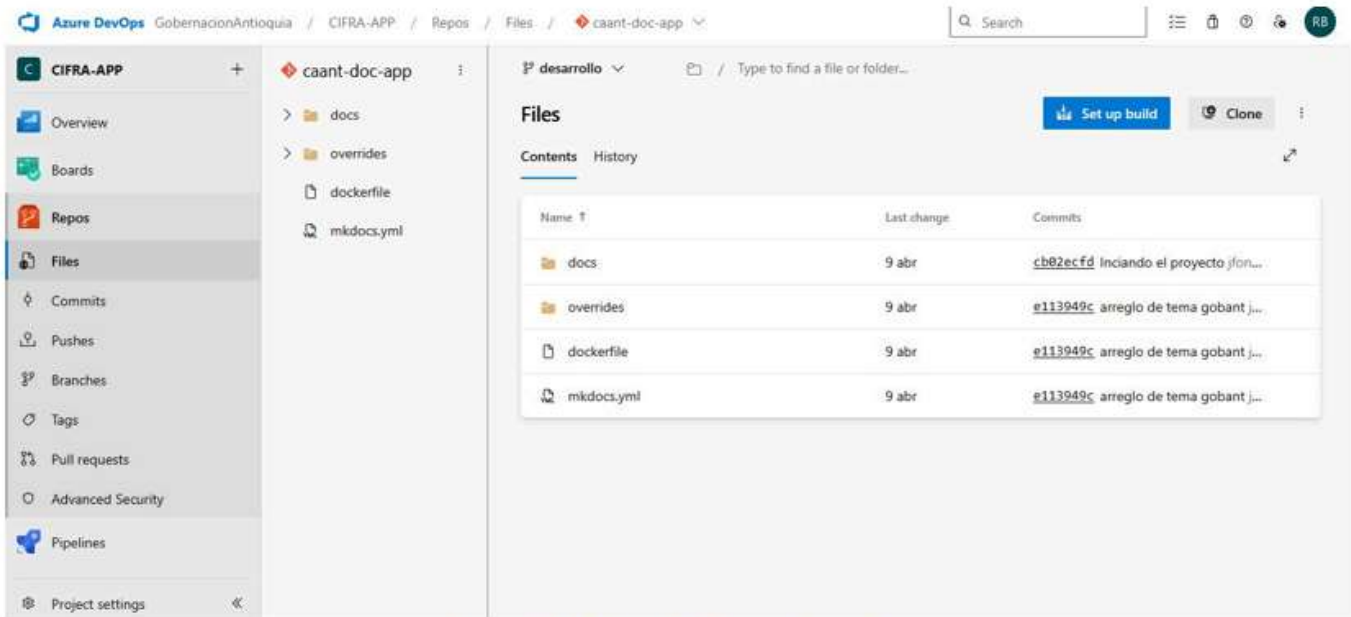


Figura 10. Obtener URL para hacer Clone.

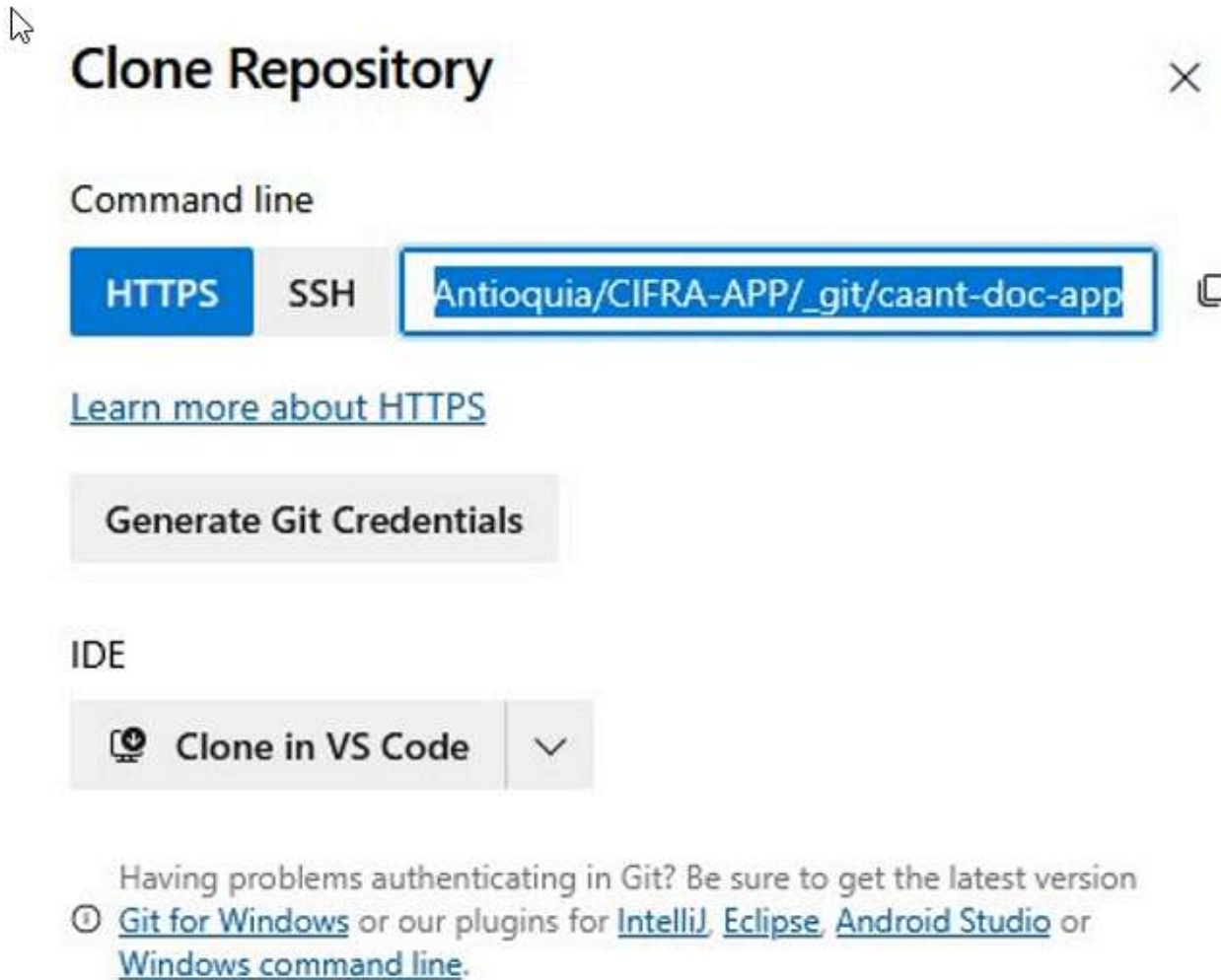


Figura 11. Opciones para clonar.

En la imagen anterior se Azure DevOps nos permite copiar la URL de Clonación o iniciar el proceso directamente en el IDE, a través del botón Clone in XXX este despliega la lista de IDEs compatibles para el proceso.

3.2. Commit & Push

PROCESO DE PUBLICACIÓN

Dos pasos: **Commit** (repositorio local) → **Push** (repositorio remoto)

El proceso para hacer la publicación de un cambio en Azure DevOps se compone de dos pasos, el primero es ejecutar el comando commit en el repositorio local, después de esto se debe ejecutar el comando push el cual sube el cambio al repositorio remoto. En la siguiente imagen se muestra el

detalle paso a paso de la secuencia para llevar el código al repositorio remoto, si bien el IDE ejecuta los pasos listados los eventos de Commit y Push si los debe generar el desarrollador.

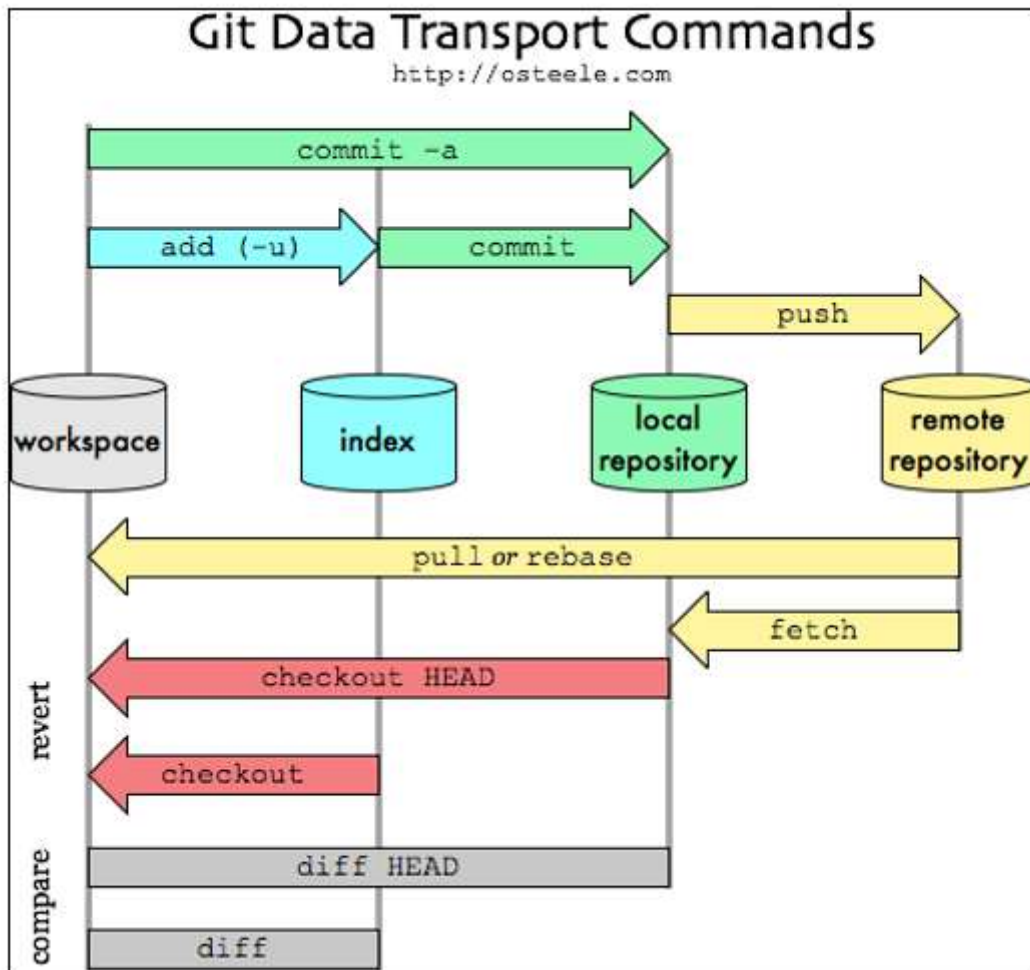


Figura 12. Secuencias de interacción con repositorio remoto.

3.3. Crear Branch

Como se describió en la sección 2.2 el uso de ramas es obligatorio para el desarrollo usando la metodología DevOps, para la creación de reamas se puede hacer desde la consola de comandos o desde Azure DevOps así:

Desde consola:

```
git branch nombre/branch
```



Este comando crea rama localmente. Necesario hacer push para crear en repositorio remoto.

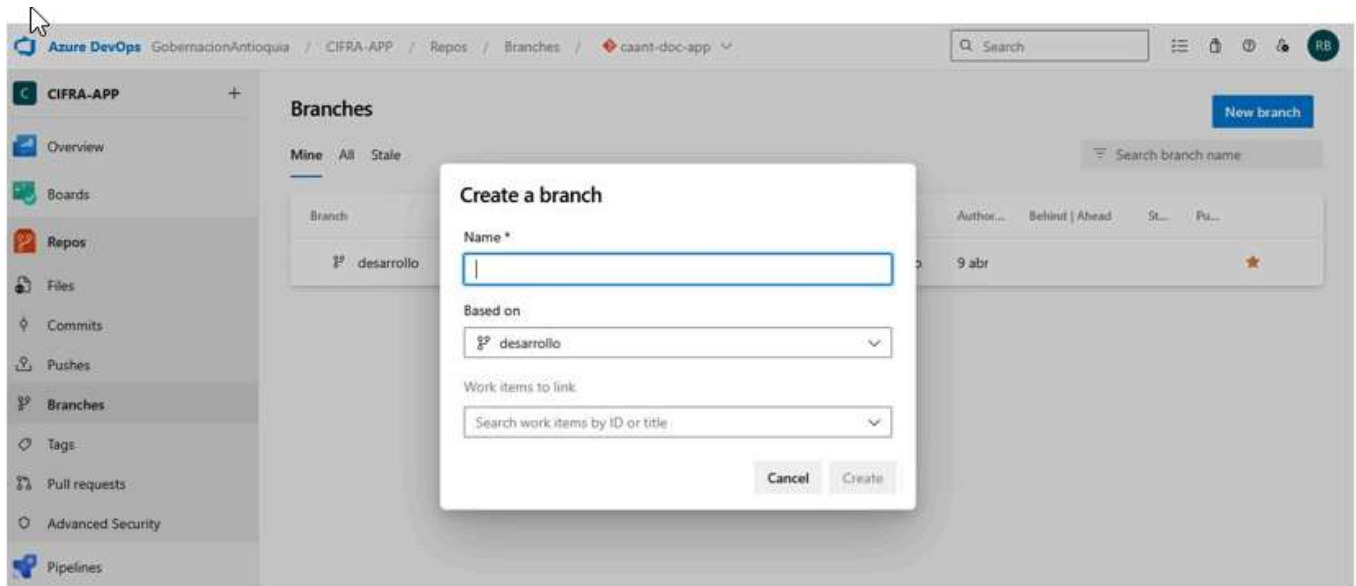


Figura 13. Creación de ramas en Azure DevOps.

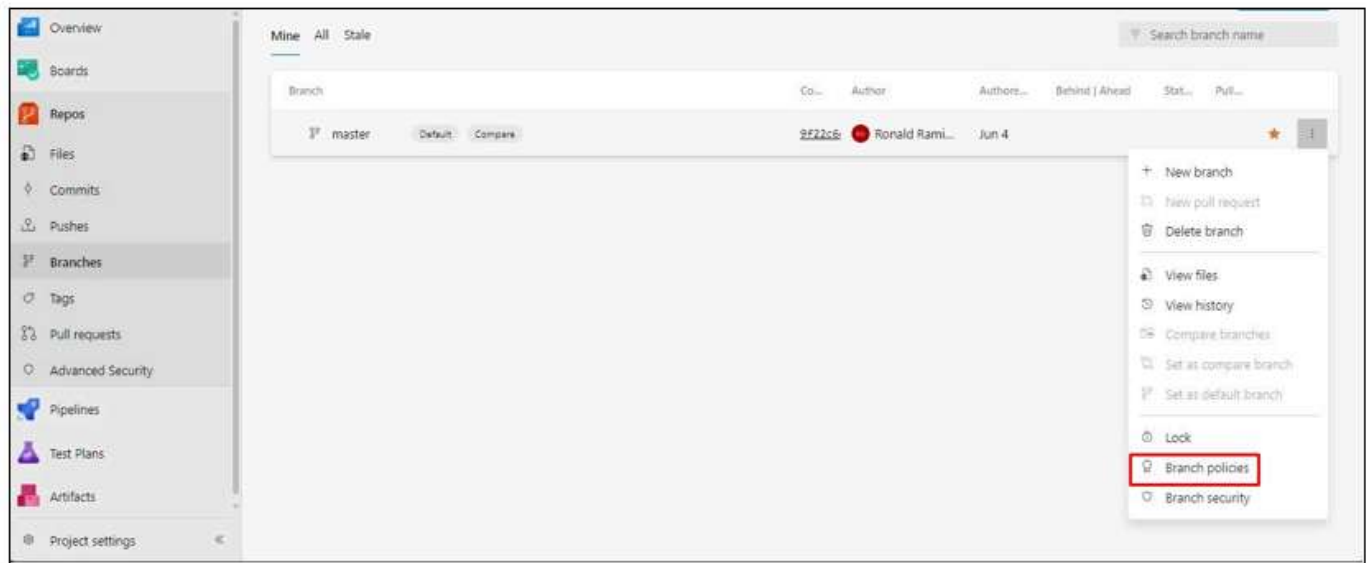
Desde Azure DevOps: Repos → Branches → **New Branch**

Para la creación de ramas en el repositorio remoto, dentro de Azure DevOps, en la opción Repos -> Branches, se habilita la opción New Branch, la cual hace la creación de la rama, para esto pide el nombre, se debe seguir el lineamiento de la sección 2.2, basado en, depende de la rama padre de la cual se quiera sacar la copia y Work Item to Link permite asociar un Item de trabajo, para que cuando se haga el push, Azure DevOps identifique y automáticamente le cambie el estado.

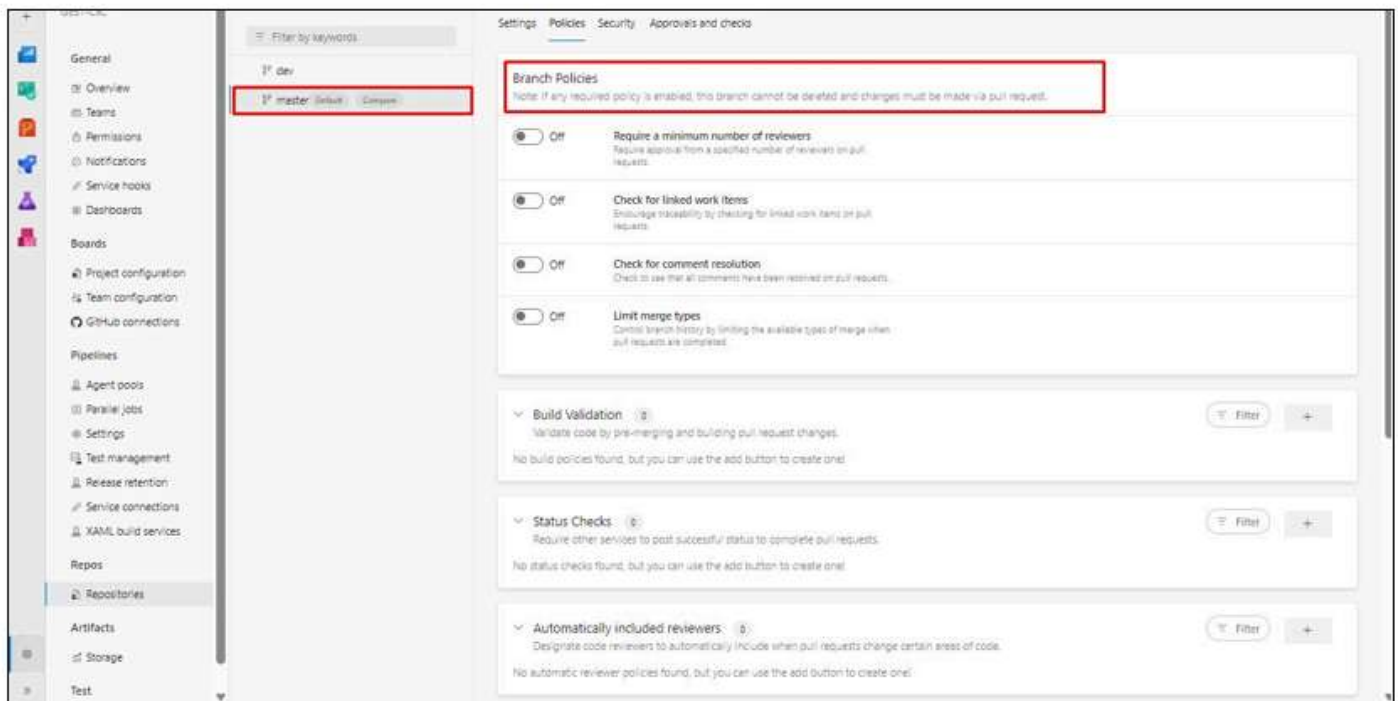
3.4. Políticas de Branch

CONTROL DE CALIDAD

Para mantener consistencia y estabilidad, establecer controles para Merge a ramas Master y Develop usando **Branch policies**.



Políticas requeridas para Master y Develop:

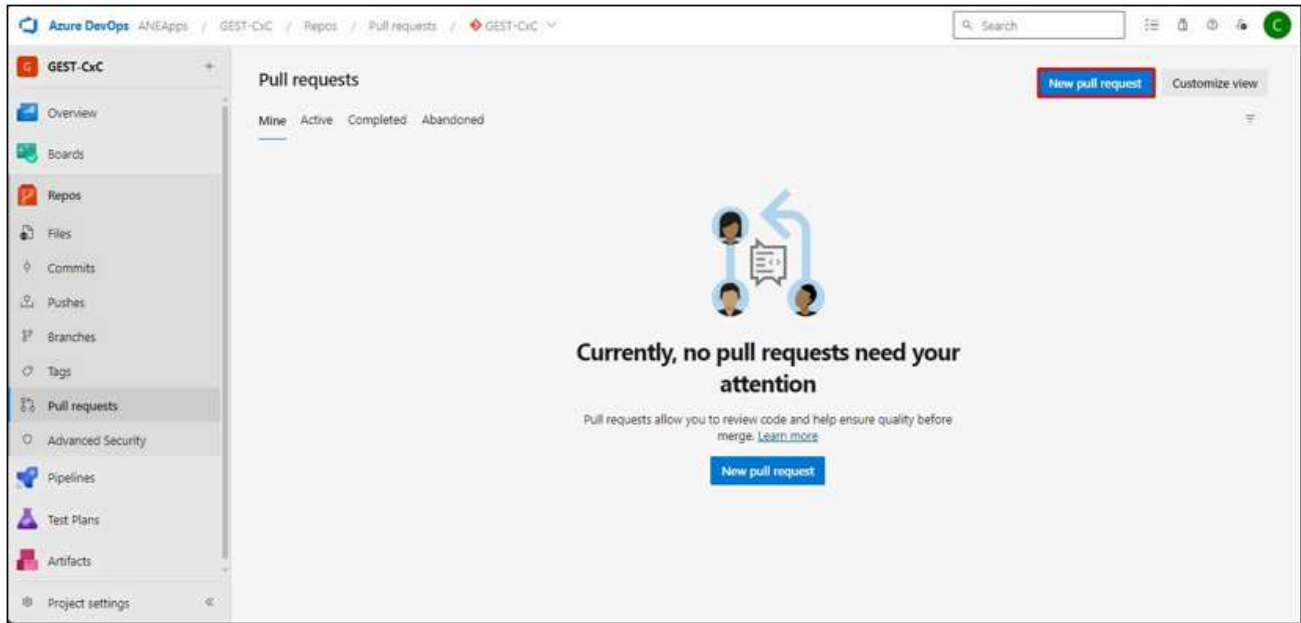


REVISIÓN DE PARES

La política de revisión se aplicará solo si el equipo tiene capacidad (perfiles y recursos) para estas revisiones.

3.5. Pull Request

Para unir ramas automáticamente: Repos → Pull Request → **New Pull request**



💡 REVISORES

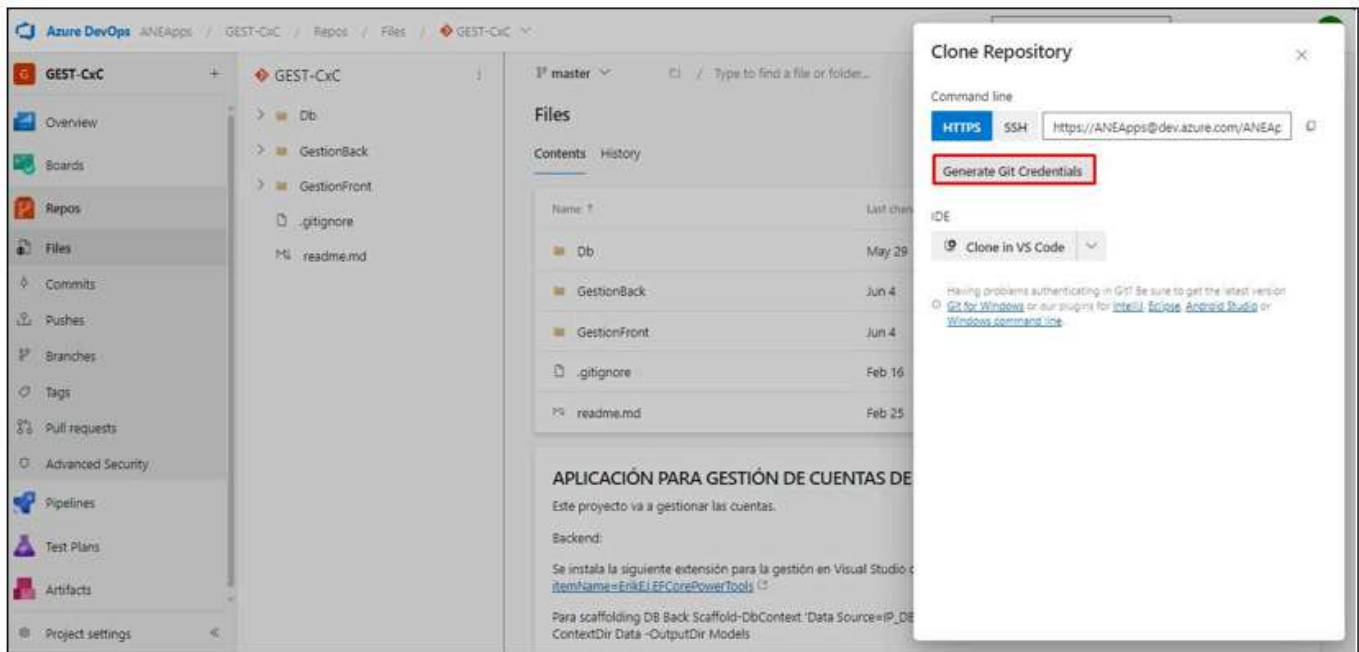
- Mínimo un revisor (configuración de política)
- Para equipos +3 desarrolladores: mínimo dos revisores
- Revisor debe ser par del desarrollador, no el mismo

El Pull Request solicitará el Branch origen y destino para hacer la unificación, una vez creado presenta los cambios que se van a unir, y pide la aprobación de los revisores, este revisor debe ser un par del desarrollador y no el mismo. Por configuración de la política se pide por lo menos un revisor, sin embargo, en los proyectos que tengan más de 3 desarrolladores, se recomienda que sean mínimo dos revisores.

🔒 3.6. Integración con Editores

Para lograr una integración sencilla desde los diferentes editores de texto se utiliza el cliente nativo de GIT, sin embargo, este cliente no tiene acceso a las credenciales de Windows, para lograr la autenticación se necesita generar unas credenciales de GIT para el usuario del dominio, de la siguiente manera:

Para autenticación con cliente GIT nativo, generar credenciales desde **Generate Git Credentials:**



4. Pruebas

! ÉXITO DE DEVOPS

La metodología DevOps basa su éxito en garantizar que cada versión que se libera cumple con un ciclo de pruebas automatizado que permite identificar errores rápidamente y de esta forma mantener la estabilidad del sistema por impacto en funcionalidades fuera del alcance del desarrollo, se recomienda tener un set de pruebas automatizado que permita hacer las pruebas de regresión del sistema, las cuales se describen a continuación.



4.1. Pruebas Unitarias

Las pruebas unitarias son las que lleva a cabo el desarrollador, estas se deben construir para todas las funcionalidades del código y se debe evaluar el cubrimiento del código desarrollado, esto significa que las pruebas unitarias garantizan que si una funcionalidad no modificada genera un resultado diferente al esperado hay que hacer un ajuste en el código no modificado, para mantener la estabilidad del sistema.

El porcentaje de cubrimiento de pruebas unitarias representa la cantidad de líneas que se prueban de forma unitaria versus las que no, este es un indicador que permite al equipo estar tranquilo con la estabilidad del sistema, a mayor porcentaje mayor estabilidad.

INDICADOR DE CUBRIMIENTO

El porcentaje de cubrimiento representa líneas probadas vs no probadas. A mayor porcentaje, mayor estabilidad.

A continuación, se presenta el listado de herramientas que se deben usar para la construcción y ejecución de pruebas unitarias.

Herramientas por Lenguaje:

LENGUAJE	PRUEBAS	CUBRIMIENTO
 JAVA	JUNIT	JACOCO
 PHP		PHPUnit
 ANGULAR	KARMA	KARMA, JASMIN, ISTANBUL
 .NET (C#, VB)	VISUAL STUDIO	VISUAL STUDIO

ANÁLISIS ESTÁTICO

Se recomienda el uso de herramientas de análisis estático de código como SonarQube, las cuales permiten identificar vulnerabilidades, errores de codificación y problemas de calidad técnica desde etapas tempranas del ciclo de desarrollo. Azure DevOps facilita significativamente esta integración gracias a sus conectores nativos y extensiones disponibles en su marketplace, lo que permite incorporar análisis automatizados en los pipelines de CI/CD de forma sencilla y eficiente.

5. Integración Continua

El proceso de integración continua (CI) se hace a través de la herramienta Azure DevOps, usando la opción de Pipelines del menú Pipelines, esta permite configurar paso a paso la secuencia de construcción, de los flujos de integración continua.

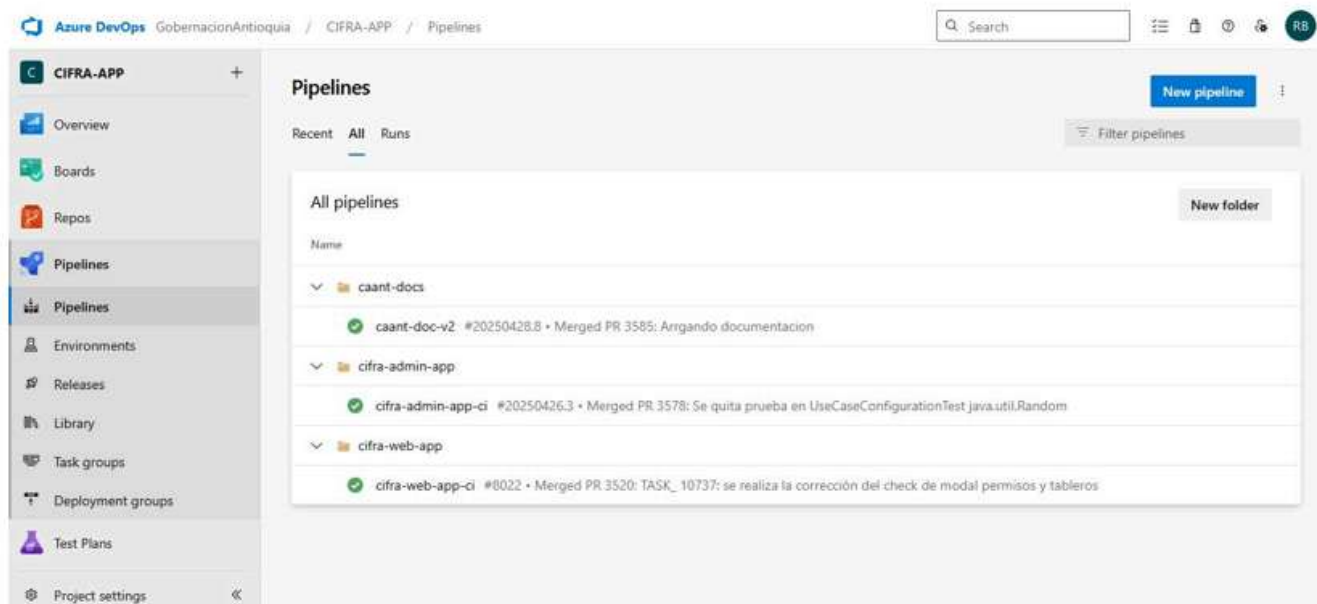
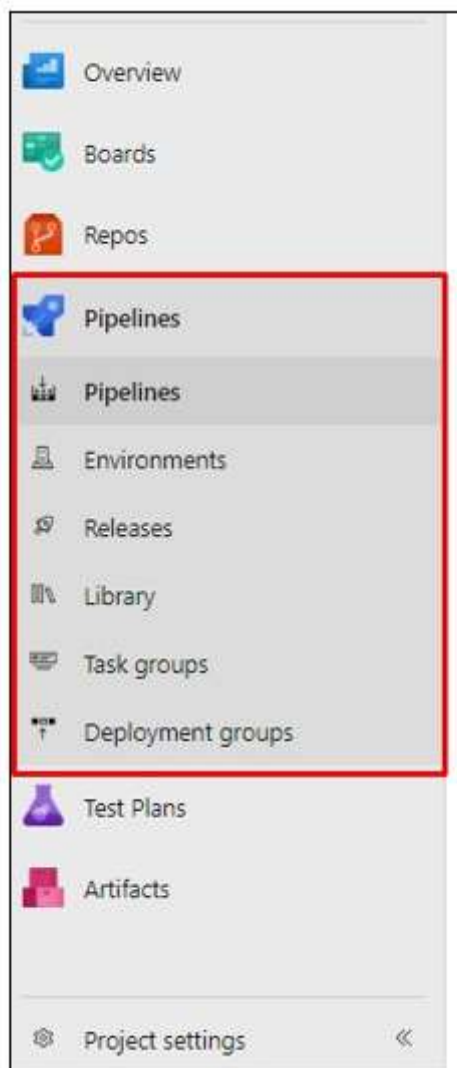


Figura 19. Crear pipeline.

5.1. Análisis Estático

Usando **SonarQube** podría ser aprovisionado por la Gobernación de Antioquia, integrado desde pipeline de construcción.

5.2. Repositorio de Artefactos

Azure Artifacts El repositorio de artefactos creados como librerías reutilizables tipo Maven para los proyectos es Azure Artifacts provisto por Azure DevOps, en ese se mantienen las versiones de las librerías usadas de forma transversal por los proyectos.

5.3. Análisis de Seguridad

RECOMENDACIÓN

Incluir **Advance Security** en repos y Pipelines creados.

5.4. Conexión a SonarQube

Pasos de configuración:

1. Preparar entorno SonarQube

- Instalar en servidor local/nube
- Asegurar disponibilidad desde Azure DevOps
- Crear token de autenticación

2. Instalar extensión en Azure DevOps

- Ir a Marketplace
- Buscar extensión SonarQube
- Instalar en organización

3. Configurar Service Connection

- Project Settings → Service connections
- New service connection → SonarQube

- o Ingresar URL y token

4. **Modificar archivo YAML del pipeline**

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

variables:
  SONARQUBE_ENDPOINT: 'SonarQubeConnection'

steps:
- task: SonarQubePrepare@5
  inputs:
    SonarQube: '$(SONARQUBE_ENDPOINT)'
    scannerMode: 'CLI'
    configMode: 'manual'
    cliProjectKey: 'mi-proyecto'
    cliProjectName: 'Mi Proyecto'
    cliSources: '.'

- task: DotNetCoreCLI@2
  inputs:
    command: 'build'
    projects: '**/*.csproj'

- task: SonarQubeAnalyze@5

- task: SonarQubePublish@5
  inputs:
    pollingTimeoutSec: '300'
```

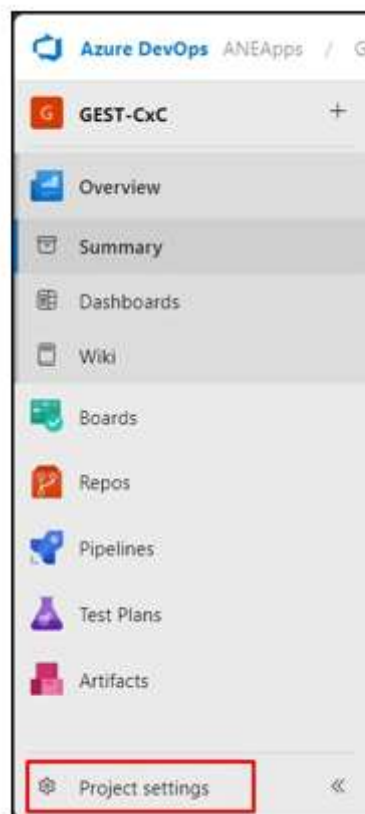



Figura 20. Configuración del proyecto.

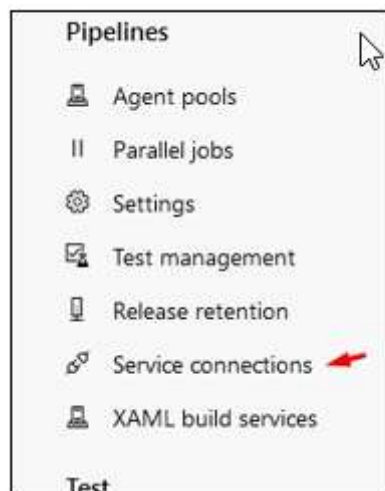


Figura 21. Conexión a servicios externos.



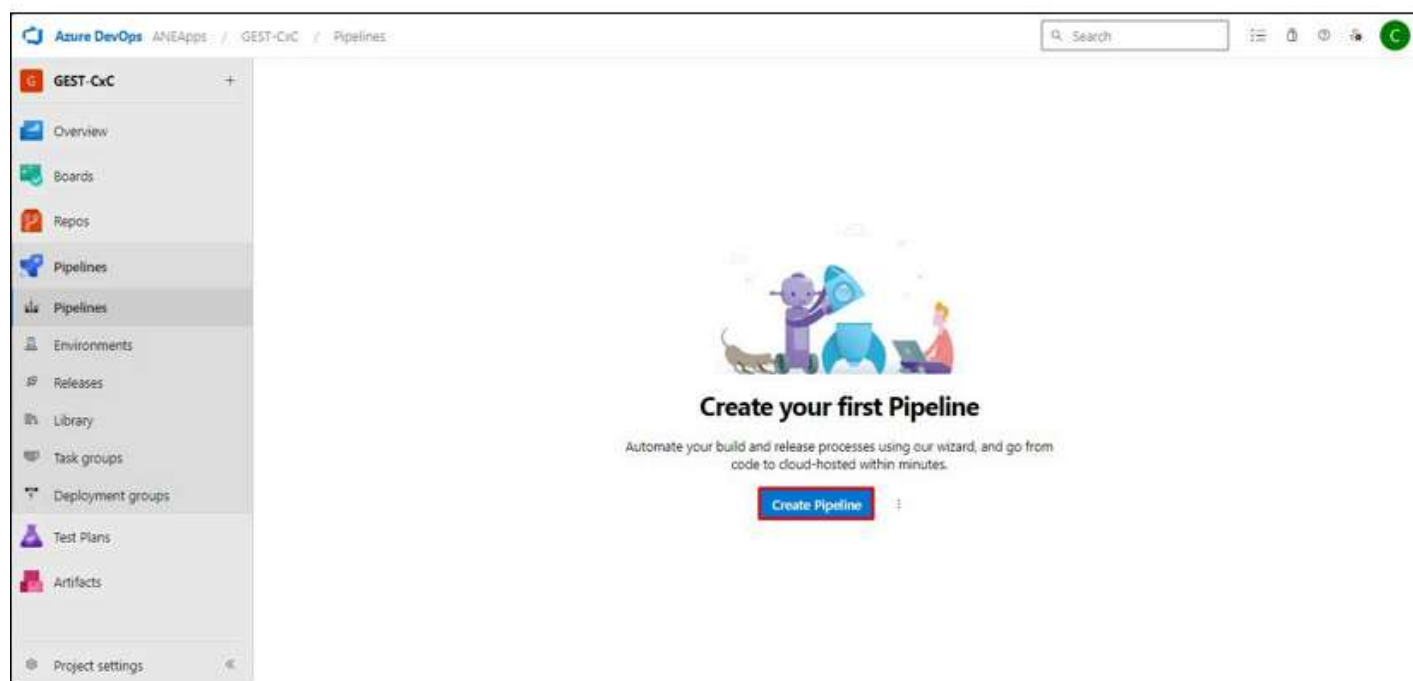
Figura 22. Nuevo servicio de conexión.



5.5. Proceso de Construcción

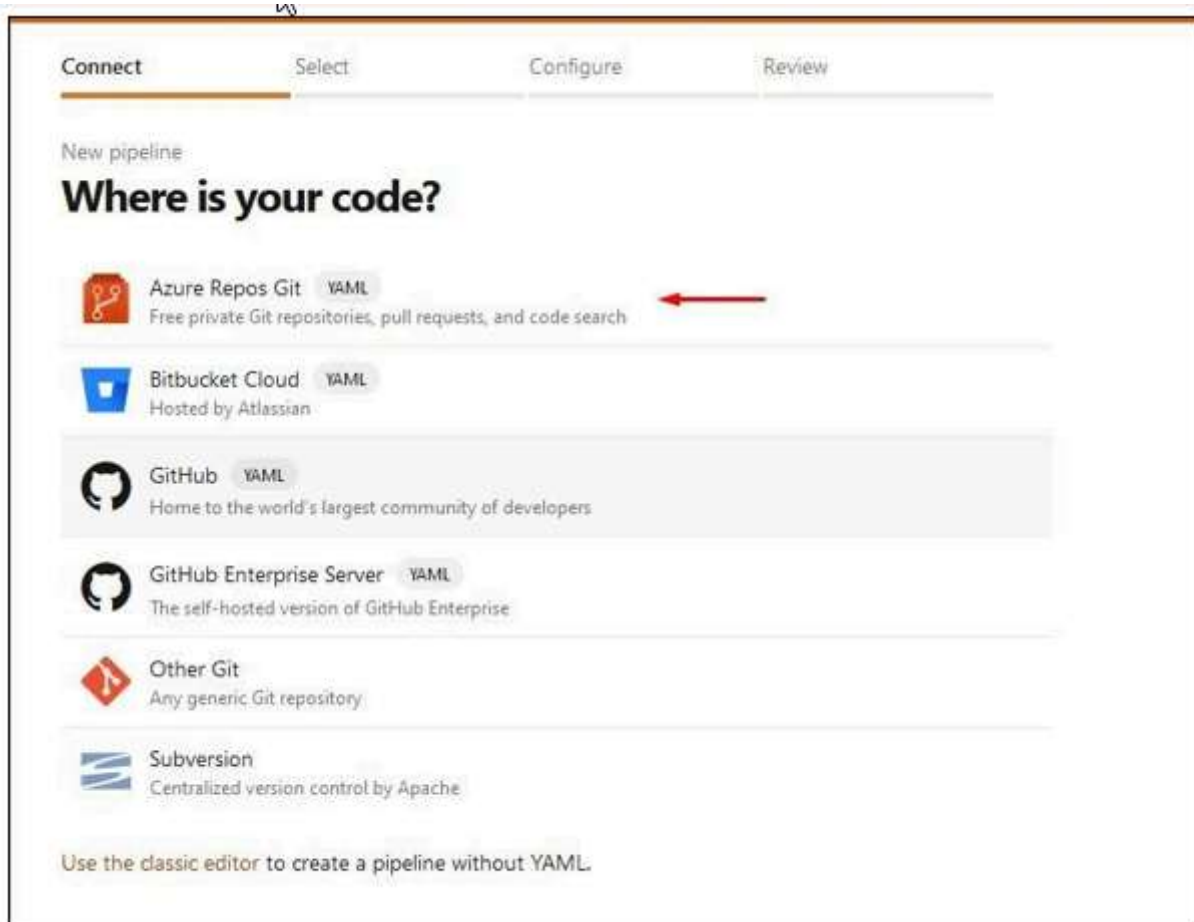
Los flujos de construcción de los proyectos de código son soportados en la herramienta Azure DevOps, a través de los Pipelines, estos permiten la creación de la secuencia de pasos de automatización de construcción. Para crear un flujo de construcción (Pipeline), se hace clic en el botón New Pipeline, este nos abre dirige a la funcionalidad para crear el flujo para el tipo de proyecto que se esté configurando.

Para crear: **New Pipeline**

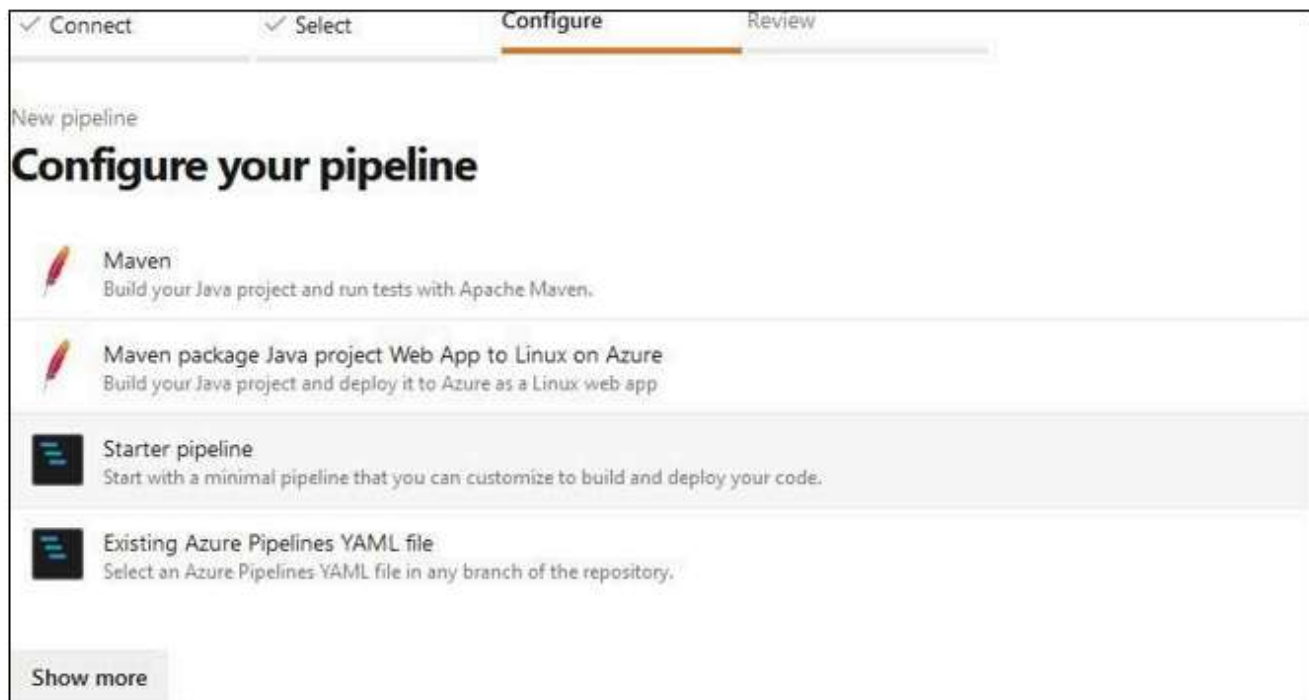


📋 Pasos:

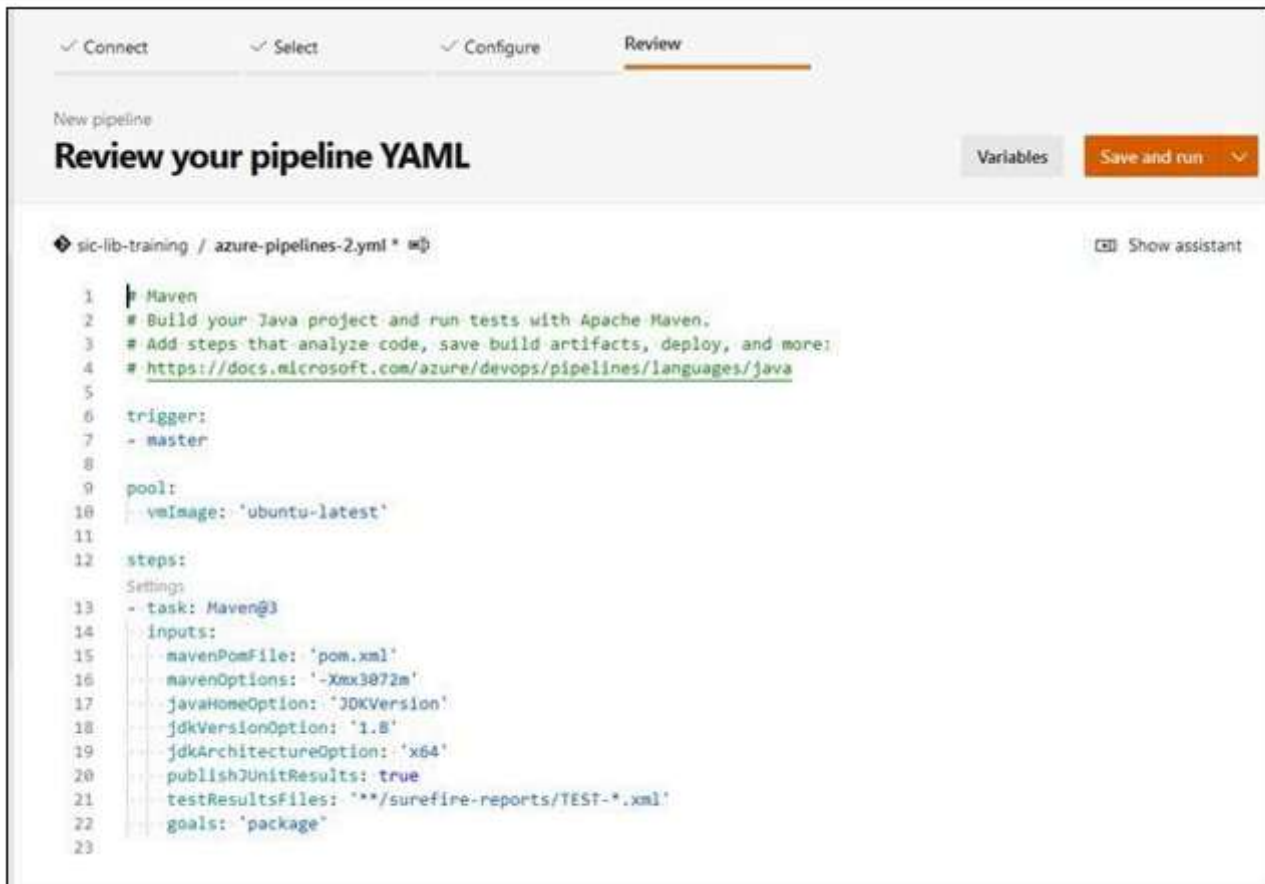
1. Seleccionar origen del código fuente



2. Azure DevOps detecta tipo de proyecto



3. Configuración YAML



```
1 # Maven
2 # Build your Java project and run tests with Apache Maven.
3 # Add steps that analyze code, save build artifacts, deploy, and more:
4 # https://docs.microsoft.com/azure/devops/pipelines/languages/java
5
6 trigger:
7   - master
8
9 pool:
10   vmImage: 'ubuntu-latest'
11
12 steps:
13   - task: Maven@3
14     inputs:
15       mavenPomFile: 'pom.xml'
16       mavenOptions: '-Xmx3872m'
17       javaHomeOption: 'JDKVersion'
18       jdkVersionOption: '1.8'
19       jdkArchitectureOption: 'x64'
20       publishJUnitResults: true
21       testResultsFiles: '**/surefire-reports/TEST-*.xml'
22       goals: 'package'
23
```



6. Despliegue Continuo

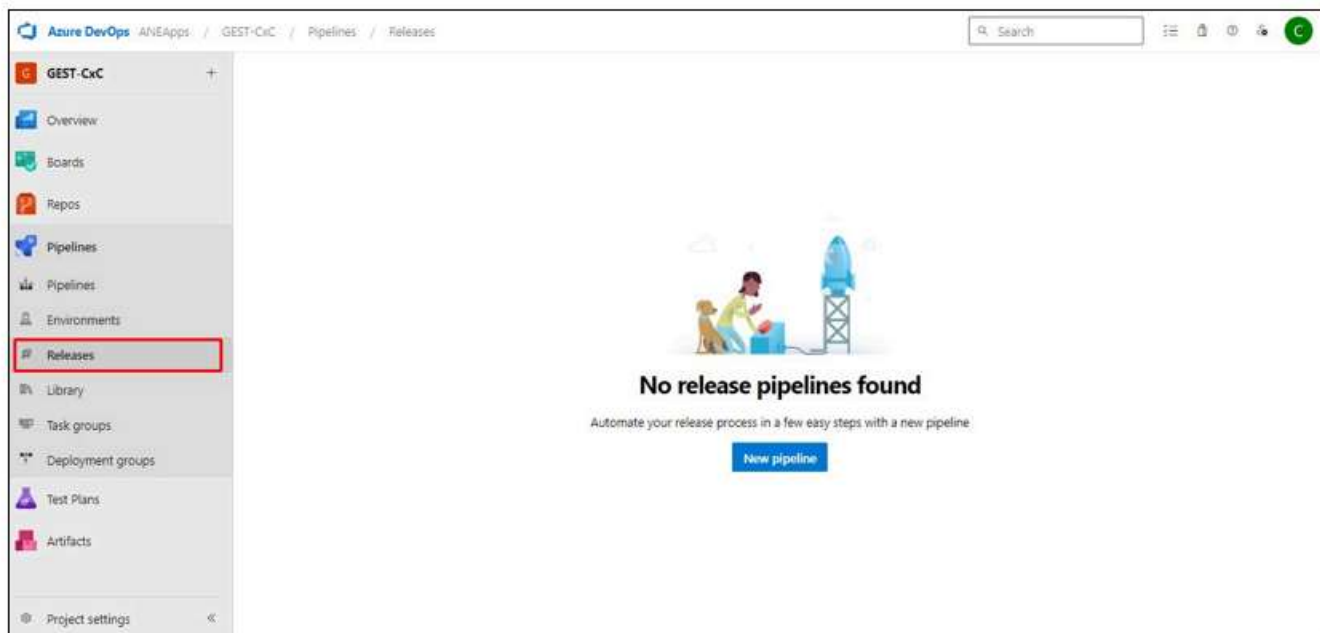
El proceso de despliegue continuo es el encargado de tomar los artefactos de construcción (CI) y desplegarlos de forma automáticamente en los ambientes dispuestos para las pruebas y operación de los sistemas, por lo tanto, es necesario que existan previamente los pipelines de integración continua y que estos publiquen los artefactos de despliegue creados en archivo YAML o de CLASSIC EDITOR.

⚠ AUTOMATIZACIÓN DE DESPLIEGUE

Proceso que toma artefactos de construcción (CI) y los despliega automáticamente en ambientes de pruebas y operación.

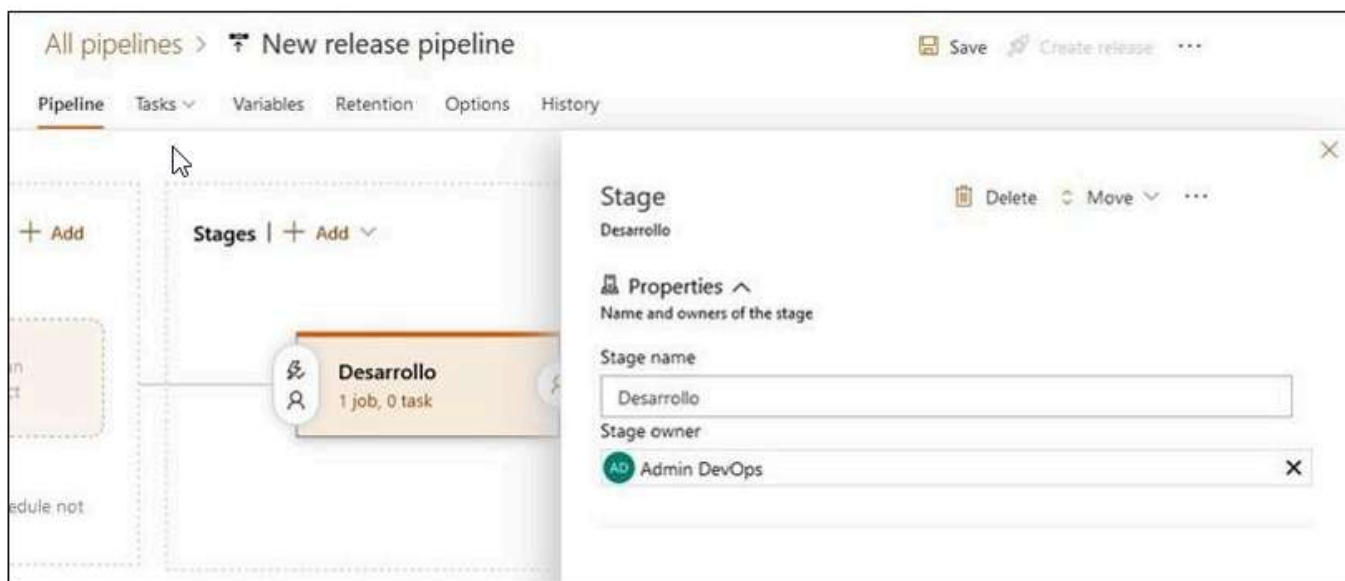
⚠ PREREQUISITOS

- Pipelines de integración continua existentes
- Artefactos publicados en YAML o Classic Editor

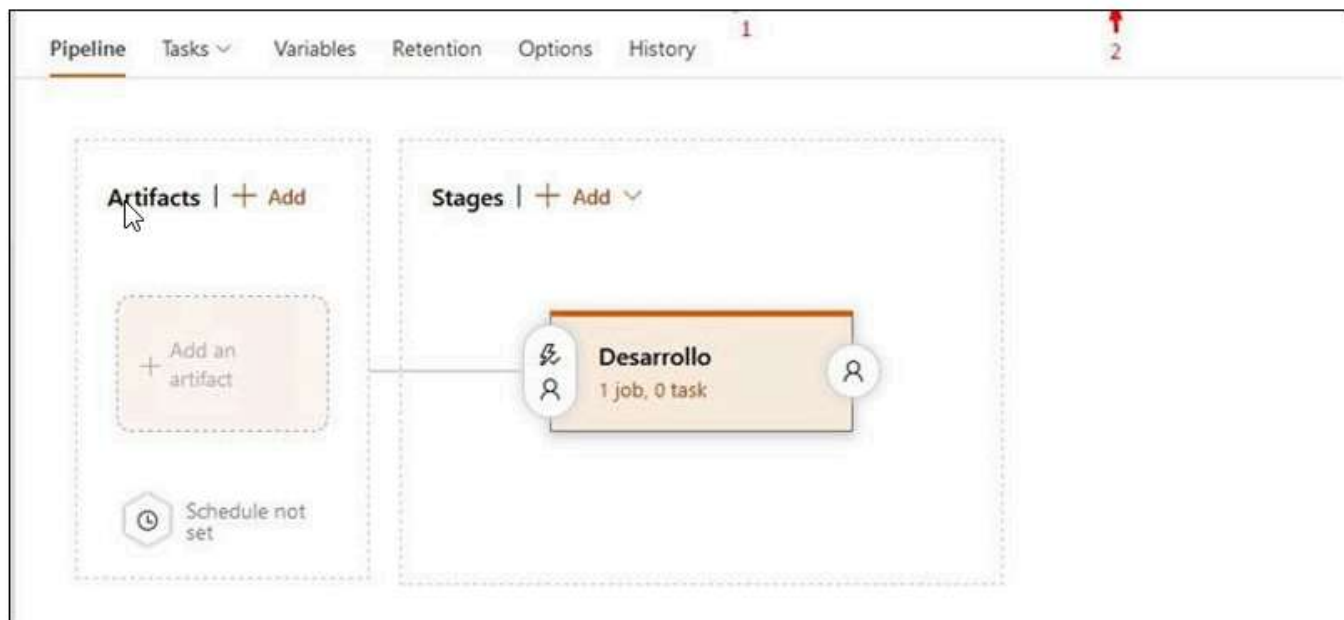


Pasos para crear Pipeline de despliegue:

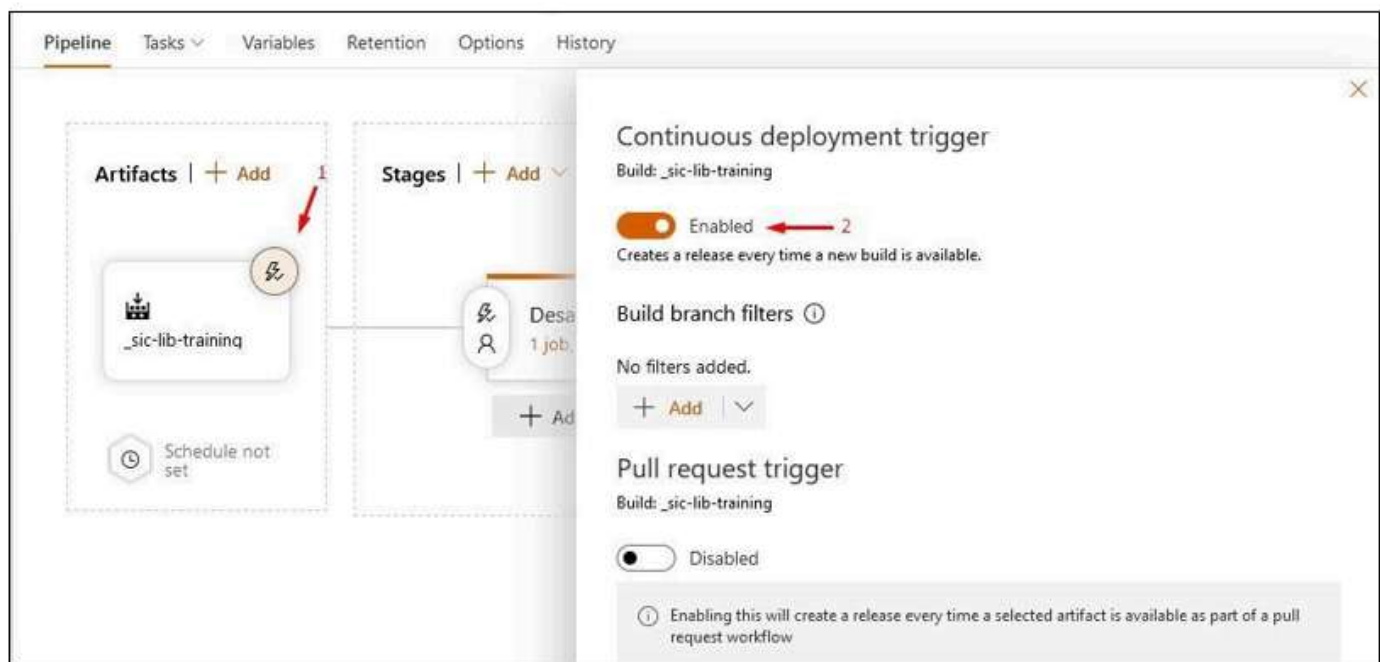
1. Crear primer Stage



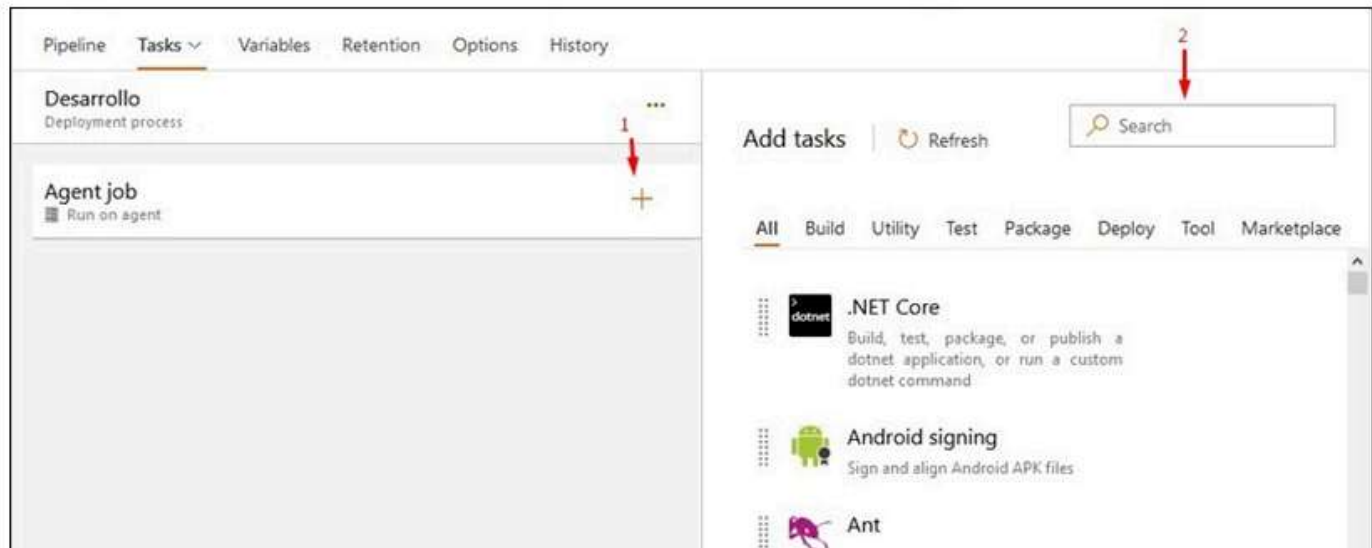
2. Nombrar ambiente (ej: Desarrollo para rama develop)



3. Configurar trigger automático

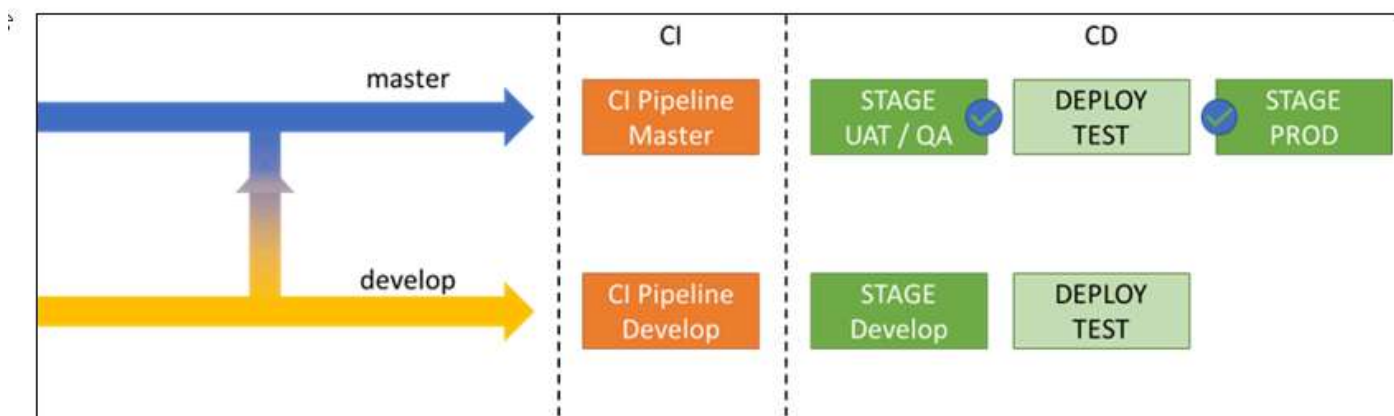


4. Agregar pasos específicos



6.1. CI Exclusivo

Corresponde al esquema en el cual cada rama tiene si propio pipeline de integración continua, por lo que en consecuencia el pipeline de despliegue continuo toma el artefacto correspondiente a la construcción específica de la rama.



7. Referencias

-  DevOps LifeCycle
-  Infografías GitFlow - Atlassian
-  Información GitFlow - Medium
-  Información TDD
-  Versiones de software - Wikipedia
-  Commit y Push - Stack Overflow

-  [Guía detallada GIT](#)
-  [Bext SA](#)

 [Editar esta página](#)